



Cleaning and Preparing Data with Pandas

The Rhode Island Traffic Stops Dataset

Real records from the Stanford Open Policing Project

Nimra Ayaz

Data Scientist & Machine Learning Engineer

Part 1 of a pandas fundamentals walkthrough

THE DATA



Meet the traffic stop records

- Each row describes one real traffic stop made by Rhode Island state police
- The file is published by the Stanford Open Policing Project as open data
- Five real rows are used here so every step is easy to trace by hand
- The exact same commands run on the complete file with `pd.read_csv`

91,741

real stops in the complete file

15

columns before cleaning

STEP 1



Take a first look at the data

```
import pandas as pd
stops = pd.read_csv('police.csv')
stops.head()
```

`read_csv` loads the real file into a DataFrame, and `head` shows the first few rows so you can see what you are working with

stop_date	stop_time	county_name	driver_gender	violation
2005/01/04	12:55	None	M	Equipment
2005/01/23	23:15	None	M	Speeding
2005/02/17	04:15	None	M	Speeding
2005/02/20	17:15	None	M	Other
2005/02/24	01:20	None	F	Speeding

These are five real rows from the file. None marks a value missing from the record

STEP 2



Find where values are missing

```
stops.isnull()
```

stop_date	stop_time	county_name	driver_gender
False	False	True	False
False	False	True	False
False	False	True	False

isnull checks every cell and returns True wherever a value is missing, and False everywhere a value is present

STEP 2



Count the missing values per column

```
stops.isnull().sum()
```

sum treats True as 1 and False as 0, so this adds up how many values are missing in each column, across the complete 91,741 row file

```
state          0
stop_date      0
stop_time      0
county_name    91741
driver_gender   5205
driver_race    5202
violation_row  5202
violation      5202
```

```
search_conducted  0
search_type      88434
stop_outcome      5202
is_arrested       5202
stop_duration     5202
drugs_related_stop  0
district          0
```

STEP 3



Drop a column that is always empty

county_name is missing for every single stop, and state never varies, since this file covers Rhode Island alone. Neither carries any signal

```
stops.drop(  
    ['county_name', 'state'],  
    axis='columns', inplace=True)
```

15

columns before

becomes

13

columns after

Passing a list drops both columns in one call. axis='columns' tells pandas to remove columns, not rows

STEP 4



Drop rows missing a key field

driver_gender is central to this analysis, so stops without it are removed rather than left in with a placeholder

```
stops.dropna(  
    subset=['driver_gender'],  
    inplace=True)
```

91,741
rows before

becomes

86,536
rows after

dropna removes any row where the listed columns are missing. Without subset, it would check every column instead

GOOD PRACTICE



A few guidelines for handling missing data



Look before you remove anything

Check the counts first so every drop is a deliberate choice



Drop a column with no signal

Empty columns and columns that never change both qualify



Drop rows sparingly

Only remove rows for fields the current question actually needs



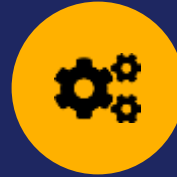
Consider filling instead of dropping

A reasonable default value can sometimes preserve the row



Your turn: clean up the missing values

Open a notebook and try each step on your own data



SECTION 2

Using the right data types

Each column has a type, and the right type unlocks the methods that make analysis easier

STEP 1



Check the type of every column

```
stops.dtypes
```

dtypes lists the storage type pandas is using for each column right now, before any fixes are made

```
stop_date      str
stop_time      str
driver_gender   str
driver_race     str
violation_raw   str
violation       str
search_conducted bool
```

```
search_type     str
stop_outcome    str
is_arrested     str
stop_duration   str
drugs_related_stop bool
district        str
```



Common pandas data types

Type	Holds	Good for
str	Text values	Names, categories, and other text data
int64	Whole numbers	Counting and simple arithmetic
float64	Numbers with decimals	Measurements, prices, and averages
bool	True or False	Filters and logical checks
datetime64	Calendar dates and times	Sorting and time based grouping
category	A fixed set of labels	Repeated text values, using less memory

pandas now labels plain text columns as str by default. The older name object still appears for columns that mix different kinds of values

WHY IT MATTERS



Why the right type makes a difference



int and float

Turn on addition, averages, and every other mathematical operation



datetime

Unlocks the year, month, weekday, and other calendar based attributes



category

Stores repeated labels more efficiently and speeds up grouping



bool

Supports filtering and logical operations directly on the column

Storing everything as text works, but it leaves these tools on the table

STEP 2



Fix a column that has the wrong type

stop_outcome	is_arrested
Citation	False
Citation	False
Citation	False
Arrest Driver	True
Citation	False

is_arrested holds the real words True and False as text, not the actual Boolean values pandas can act on

```
print(stops.is_arrested.dtype)  
str  
  
stops['is_arrested'] =  
    stops.is_arrested.astype(bool)
```

```
print(stops.is_arrested.dtype)  
bool
```



Your turn: fix a column's data type

Open a notebook and try each step on your own data



SECTION 3

Building a time based index

Combining the date and time columns unlocks calendar aware filtering and grouping

STEP 1



Combine the date and time columns

`stop_date` and `stop_time` are separate text columns. `str.cat` joins them into one column, with a space between the two parts

```
combined = stops.stop_date.str.cat(  
    stops.stop_time, sep=' ')
```

The result is still plain text. The next step turns it into an actual datetime value

```
combined  
0    2005/01/04 12:55  
1    2005/01/23 23:15  
2    2005/02/17 04:15  
3    2005/02/20 17:15  
4    2005/02/24 01:20
```

```
Name: stop_date, dtype: str
```

STEP 2



Convert the text into a datetime

```
stops['stop_datetime'] =  
    pd.to_datetime(combined)
```

to_datetime reads the text and builds a proper calendar aware value, ready for date based work

column	type
stop_date	str
stop_time	str
is_arrested	bool
stop_datetime	datetime64[us]

STEP 3



Set the new column as the index

```
stops.set_index(  
    'stop_datetime', inplace=True)
```

stop_datetime	driver_gender	violation
2005/01/04 12:55:00	M	Equipment
2005/01/23 23:15:00	M	Speeding
2005/02/17 04:15:00	M	Speeding
2005/02/20 17:15:00	M	Other
2005/02/24 01:20:00	F	Speeding

With `stop_datetime` as the index, each row can be located and grouped by when the stop happened. The original columns are still there too

WHY IT HELPS



What a DatetimeIndex makes possible



Slice by date range

Select any span of records with `loc`, such as a single month



Group by time period

Summarize totals by day, week, or month with `resample`



Pull out calendar parts

Read the hour, weekday, or month straight from the index



Sort in order automatically

Rows line up chronologically without any extra sorting step

Three steps to a clean, ready to analyze table



1

Handle missing values

Count what is missing, then drop or fill it with intent



2

Set proper data types

Convert text into numbers, booleans, and dates



3

Build a time based index

Combine date and time, then index the table by it

From here, the same real, cleaned table of Rhode Island stops is ready for grouping, comparing, and charting